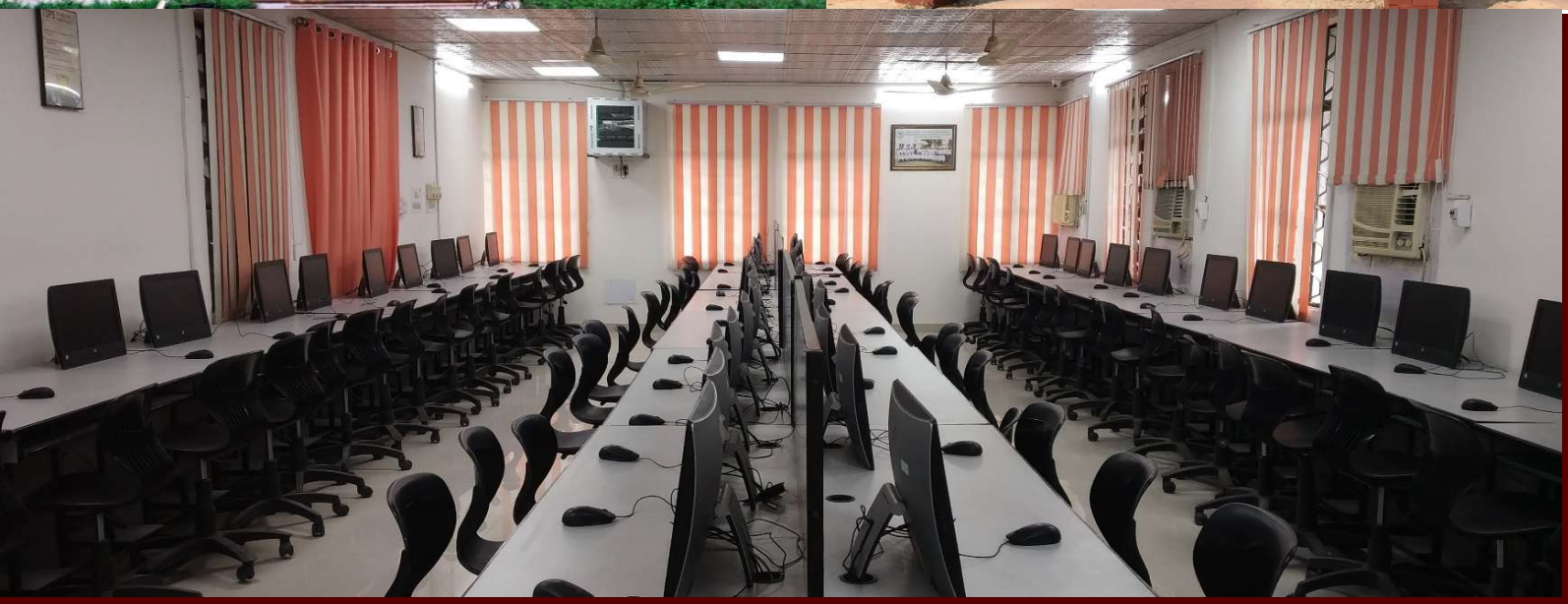
جب کوئی قوم فن اور علم سے عاری ہو جاتی ہے تو وہ غربت کو دعوت دیتی ہے اور جب غربت آتی ہے تو وہ ہزاروں جرائم کو جنم دیتی ہے۔

"When a nation becomes devoid of art and learning, it invites poverty and when poverty comes it brings in its wake thousands of crimes."
-Sir Syed Ahmad Khan

B.Sc. (Hons) CA II Semester



Laboratory Course-II

Course Code- CABSMJ2P02

DEPARTMENT OF COMPUTER SCIENCE

ALIGARH MUSLIM UNIVERSITY, ALIGARH

2025-2026

Credits

The following lab manual up-gradation committee updated the lab manual:

- ***Prof. Arman Rasool Faridi (Chairperson)***
- ***Prof. Mohammad Ubaidullah Bokhari***
- ***Prof. Aasim Zafar***
- ***Prof. Suhel Mustajab***
- ***Dr. Faisal Anwar***
- ***Dr. Mohammad Sajid***
- ***Dr. Mohammad Nadeem***
- ***Dr. Faraz Masood***

The following committee members originally designed the lab manual:

- ***Prof. Mohammad Ubaidullah Bokhari***
- ***Dr. Arman Rasool Faridi***
- ***Dr. Faisal Anwer***
- ***Prof. Aasim Zafar (Convener)***

Design & Compilation:

- ***Dr. Faraz Masood***

Revised Edition: Jan, 2026

Department of Computer Science, A.M.U., Aligarh (U.P.) India

COURSE TITLE: Lab -II **COURSE CODE: CABSMJ2P02**
CREDIT: 02 **PERIODS PER WEEK:** 03
CONTINUOUS ASSESSMENT: 60 **EXAMS:** 40

COURSE DESCRIPTION

This manual is intended for B.Sc. II Semester students in Data Structure using C. This manual typically contains practical/Lab Sessions related to Data Structure using C, covering various aspects to enhance subject understanding. In computer science, a data structure is a way of organizing and storing data in a computer program so that it can be accessed and used efficiently. Data structures manage large amounts of data, enabling efficient searching, sorting, insertion, and deletion.

Data structures can be categorized into two types: primitive data structures and non-primitive data structures. Primitive data structures, such as integers, floating-point numbers, characters, and Booleans, are the most basic in a programming language. Non-primitive data structures are complex data structures built using primitive data types, such as arrays, linked lists, stacks, queues, trees, and hash tables.

The choice of the data structure for a particular task depends on the type and amount of data to be processed, the operations that need to be performed on the data, and the efficiency requirements of the program. Efficient use of data structures can significantly improve the performance of a program, making it faster and more memory efficient. A data structure is a particular way of

organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks.

The lab manual will provide step-by-step instructions on completing the exercises, sample code, and expected output. By the end of the course, students should have a solid understanding of the principles of data structures and algorithms and be able to apply this knowledge to solve real-world problems using the C programming language.

Students are advised to thoroughly review this manual rather than only the topic mentioned in the syllabus, as practical aspects are the key to understanding and conceptualizing the theoretical aspects covered in the books.

COURSE CONTENT

This course is designed to allow the students to learn Data Structure using C. This course is intended to develop a deep understanding of various operations on data structure, such as searching, sorting, insertion, deletion, and traversing. Apart from basic concepts, it helps to understand the concept of arrays, linked lists, stacks, queues, trees, and hash tables.

OBJECTIVES

This course is designed to help students in:

- ❒ Learning basic concepts of Data structures.
- ❒ Developing data structures using C programming.

OUTCOMES

After completing this course, the students would be able to:

- ❒ Understand and implement the different data structures using C programming.
- ❒ Understand and implement basic operations using C programming.
- ❒ Write, debug, and execute programs to solve various data structure problems.

RULES AND REGULATIONS

Students are required to adhere to the following rules strictly.

- ❒ The students must complete the weekly activities/assignments well in time (i.e., within the same week).
- ❒ The students must maintain the Lab File of their completed activities/assignments in the prescribed format (**Appendix-1**).
- ❒ The students must get the completed weekly activities/assignments checked and signed by the concerned teachers in the lab in the immediately succeeding week. Failing which, activities/assignments for that week will be treated as incomplete.
- ❒ At least **TEN (10)** such timely completed and duly signed weekly activities/assignments are compulsory, failing which students will not be allowed to appear in the final Lab Examination.
- ❒ The students need to submit the following deliverables for each exercise, duly signed by the concerned teacher:
  - ❖ Coding
  - ❖ Input /Output
- ❒ Each question will be evaluated on a scale of 10 points, 7 for Coding and 3 for Input/Output.

- The students must ensure that each question is assessed and signed by the concerned Teacher in the week/time.
- Late submissions will not be accepted after the due date.
- Cooperate, collaborate, and explore for the best individual learning outcomes, but copying is strictly prohibited.
- The Continuous Lab assessment will be based on two sessionals, each carrying 30 marks.
- Marks distribution for each sessional:
  - 20 Marks: For a duly signed lab report by the respective lab teacher.
  - 5 Marks: For solving a lab question/program on the day of the sessional.
  - 5 Marks: For the viva conducted during the sessional.
- This distribution ensures a balanced evaluation of students' practical work, problem-solving skills, and conceptual understanding.

APPENDIX-1

Template for the Index of Lab File

WEEK NO.	PROBLEMS WITH DESCRIPTION		PAGE NO.	SIGNATURE OF THE TEACHER WITH THE DATE
1	1#			
	2#			
	3#			
2	1#			
	2#			
	3#			
3	1#			
	2#			
	3#			
4	1#			
	2#			
	3#			

Note: The students should use the Header and Footer of each page to mention their roll no. & name.

WEEK #1

Objectives

- ❒ *To learn the concept of an array and string in C.*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the basic features of the Array to manipulate the string.*

Problems

1# Write a C program to remove all vowels from a string.

Test: Empty string: "" → "".

No vowels: "rhythm" → "rhythm".

Only vowels: "aeiouAEIOU" → "".

Mixed case + spaces: "Hello World" → "Hll Wrld".

With digits/symbols: "C0d!ng" → "C0d!ng".

2# Write a C program to remove a given word from a string.

Test: Word present once: "I love C", word="love" → "I C"

Word not present: "I love C", word="Java" → "I love C".

Word appears multiple times: "test test test", word="test" → "" or " " (define spacing
rule).

Word at start/end: "hello world", word="hello" → " world"; "hello world", word="world" → "hello ".

Case sensitivity: "Hello", word="hello" → unchanged if comparison is case-sensitive.

3# Write a C program to find the number of vowels, consonants, digits, and white space character

Test: Empty string: all counts 0.

Only letters: "abcXYZ" → vowels=2, consonants=4.

Only digits: "12345" → digits=5.

Only spaces/tabs: " \t " → whitespace>0, others 0.

Mixed: "Hello 123" → vowels=2, consonants=3, digits=3, whitespace=1.

Non-alphanumerics: "@#" → all counts 0 except maybe "others" if implemented.

4# Write a C program to perform the following operation in a matrix

a. Addition b. Subtraction c. Multiplication d. Transpose

Test: Addition/Subtraction:

a. 2×2 matrices with small ints.
b. Zero matrix + non-zero matrix.

Multiplication:

a. 2×2 identity × matrix = matrix.
b. 2×3 × 3×2 with known result.
c. Matrix × zero matrix = zero matrix.

Transpose:

a. 1×n row → n×1 column.
b. Symmetric matrix: transpose equals original

5# Write a C program to perform the following operation on strings using string functions.

a. Addition b. Copying c. Reverse d. Length of String.

Test: Addition (concatenation):

a. "Hello" + "World" → "HelloWorld".
b. Empty + non-empty; non-empty + empty.

Copying:

a. Copy non-empty to empty buffer; copy empty string.

Reverse:

a. "abcd" → "dcba"; single char "a" → "a".

Length: "" → 0; "Hello" → 5.

WEEK #2

Objectives

- ❒ *To learn the basic concept of Structures in C Programming*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the basic features of the Structures.*

Problems

1# Write a C program to store students' information on using a structure.

2# Write a C program to add two distances (in inch-feet) system using structure.

Test: Inches sum ≥ 12: (5 ft 9 in) + (4 ft 6 in) → (10 ft 3 in)

Zero distance + non-zero distance.

3# Write a C program to add two complex numbers by passing structure to a function.

Test: One zero: (0+0i) + (5−2i) → (5−2i).

Negative parts: (−2+3i) + (4−5i) → (2−2i)

4# Write a C program to calculate the difference between the two time periods (HH: MM: SS).

Test: Borrowing seconds/minutes: 10:10:05 − 09:59:50.

Crossing hour boundary: 02:00:00 − 01:30:00.

5# Write a C program to store information using structures and pointers with dynamic memory allocation.

WEEK #3

Objectives

- ❒ *To learn the complex problem of the Array.*

Outcomes

After completing this, the students would be able to:

- ❒ *Learn how to perform various operations on the Array.*

Problems

1# Write a C program that uses functions to perform the following operations on an array

a. Create an array of size 25 and insert the element of your choice
b. Delete the element at the beginning, middle, and end and adjust the Array.
c. Insert the element at the particular locations in the Array.
d. Display all the elements

2# Write a C Program to merge two sorted arrays into one sorted array

Test: Same size arrays: [1,3,5] + [2,4,6] → [1,2,3,4,5,6].

Unequal sizes: [1,2] + [3,4,5,6]. → [1,2,3,4,5,6]

One empty, one non-empty.

Duplicate values: [1,2,2] + [2,3] → [1,2,2,2,3]

All elements of one array smaller than the other: [1,2,3] + [10,11] → [1,2,3,10,11]

3# Write a C Program to traverse the array using pointers.

4# Write a C Program that takes an index as input and prints the element of the array at that index using pointer arithmetic.

WEEK #4

Objectives

- ❒ *To learn the concept of various functions on a linked list.*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the linked List's concepts: creation, deletion, and insertion.*

Problems

1#. Write a C program that uses functions to perform the following operations on a singly linked list

I. Creation
II. Insertion
   - At beginning
   - At middle
   - At end
III. Deletion
   - At beginning
   - At middle
   - At end
IV. Traversal.

Optional Questions

S#1. Imagine you are building a simple shopping cart system where the cart is implemented as a **Singly Linked List**.

Each node in this list represents an item and contains:

1. itemName (String)
2. price (Integer)

3. quantity (Integer, which defaults to 1)

Write Functions to perform the following task:

Adding items to cart `*add _cart_item()*`

Delete items from the cart `*delete_cart_item()*`

Update Existing: If the item name is already inside the cart, do not create a new node (do not add the item). Instead, find that node(item) and increment its quantity by 1.

WEEK #5

Objectives

- ❒ *To learn the concept of various functions on a doubly-linked list.*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the concepts: creation, deletion, and insertion of the doubly-linked List*

Problems

1. Write a C program that uses functions to perform the following operations on a doubly linked list

1# Creation

2# Insertion

a. At beginning

b. At middle

c. At end

3# Deletion

a. At beginning

b. At middle

c. At end

4# Traversal.

Optional Questions

S#2. You are building the navigation feature for a Music Player application (like Spotify). The

playlist is implemented as a **Doubly Linked List** so that a user can easily press "Next Song" or "Previous Song."

Each node represents a Song and contains:

- SongID
- Song Name
- next pointer
- prev pointer

The task is:

Implement functions `*next_song()*` to play the next song and `*previous_song*` to play previous song.

Write a function called `*delete_current_song(currentSongNode)*`. The user is listening to a song and decides they hate it. When this function is called:

1. Remove the currentSongNode from the list.
2. Connect the previous song directly to the next song (bridging the gap).
3. Return the node of the **next** song so the player can immediately start playing the next track.
4. *Edge Case:* Handle the scenario where the song being deleted is the first or the last song in the playlist.

WEEK #6

Objectives

- ❒ *To learn the concept of various functions on a circular linked List.*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the concepts: creation, deletion, and insertion of the circular linked List*

Problems

1# Write a C program that uses functions to perform the following operations on a circular Linked List

a. Creation
b. Insertion
c. Deletion
d. Traversal.

Optional Questions

S#1 You are building the turn-management system for a multiplayer board game (like Uno or Monopoly). The players sit in a circle, so the turns must rotate indefinitely: Player 1 -> Player 2 -> Player 3 -> Player 1 -> ...
Each node represents a Player and contains:

- PlayerID
- Player Name
- next pointer
- prev pointer

Implement it using a **Circular Linked List** where the last node's `next` pointer points back to the `head`.

The Task:
Write a function called `end_turn(currentPlayerNode)` that perofrms:
It should print "Turn ending for [Player Name]".
It should then simply return the node of the next player who needs to take their turn.
It should also print the name of the next player whose turn arrived.
Constraint: Because the list is circular, the function should never encounter a `NULL` value; it should be able to loop through the players forever.

WEEK #7

Objectives

- ❒ *To learn the implementation of the stack using the Array and linked lists*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the concepts of Creation, Push, and Pop operations on the stack.*
- ❒ *Use the stack for expression conversion.*

Problems

1# Write a C program that implements a stack and its operations using

a. Arrays

b. Linked List (Pointers).

2# Write a C program to convert the Infix expression into a Postfix expression.

3# Write a C program to evaluate a postfix expression.

4# Write a C program to convert the following infix expression into postfix form.

(A + B) * (C + B) * (E / F).

WEEK #8

Objectives

- ❒ *To learn the implementation of the Queue using the Array and linked lists*

Outcomes

After completing this, the students would be able to:

- ❒ *Explain the concepts: creation, insertion, and deletion operation on the Queue*

Problems

1# Write a C program that implements Queue and its operations using

a. Arrays

b. Linked List (Pointers).

Optional Questions

S#1 Scenario: Supermarket Checkout Line

At a local supermarket, customers arrive at the checkout counter and join a single line (queue). The cashier serves one customer at a time, following the First-In-First-Out (FIFO) principle.

Write a C program that simulates this checkout queue using a linear queue. The program should support the following operations:

1. **Add a customer** to the end of the queue (when they arrive).
2. **Serve the next customer** (remove them from the front of the queue).
3. **Display the current queue** of waiting customers.

Each customer is identified by a name and customer ID.

Task:
Write a C program using an **array-based queue** to simulate the supermarket checkout system.

S#2 Develop a module for a printer spooling system (A spooling system is a method used in computing to manage data transfer between a computer and slower peripheral devices, like printers. It temporarily holds data in a queue, allowing the computer to continue working on other tasks while the spooled data is processed in the background) that handles multiple sequential print requests. If a new request arrives while the printer is active, it must be added to a queue rather than being rejected. The system should process documents in the order they were received (First In, First Out).

Task: Implement the **queue using a Linked List**, where each node contains the *document name* and *number of pages*.

WEEK #9

Objectives

- ❒ *To learn about circular Queues.*

Outcomes

After completing this, the students would be able to:

- ❒ *Construct the circular Queue and its operations*
- ❒ *Explain various search algorithms*

Problems

1# Write a C program that implements a Circular Queue using arrays.

2# Write a C program that uses both recursive and non-recursive functions to perform the following searching operations for a key value in a given list of integers:

a) Linear search

b) Binary search.

WEEK #10

Objectives

- ❒ *To learn about the various sorting algorithms.*

Outcomes

After completing this, the students would be able to:

- ❒ *Learn the concept of sorting and its mechanisms.*

Problems

1# Write a C program that implements the following sorting algorithms.

a) Insertion sort for integers

b) Bubble sort for characters

c) Selection sort for integers

d) Quick sort for characters

WEEK #11

Objectives

- ❒ *To learn about the various sorting algorithms.*

Outcomes

After completing this, the students would be able to:

- ❒ *Learn the concept of sorting and its mechanisms.*

Problems

1# Write a C program that implements the following.

a) Counting sort
b) Merge sort
c) Heap sort

WEEK #12

Objectives

- ❒ *To learn the concept of the binary search tree.*

Outcomes

After completing this, the students would be able to:

- ❒ *Create, Insert, and Delete node in the binary search tree.*

Problems

1# Write a C program to perform the following operations:

a. Insert an element into a binary search tree.

b. Delete an element from a binary search tree.

c. Search for a key element in a binary search tree.

WEEK #13

Objectives

- ❒ *To learn the concept of the AVL tree.*

Outcomes

After completing this, the students would be able to:

- ❒ *Create, Insert, and Delete nodes in the AVL tree.*

Problems

1# Write a C program to implement the tree traversal methods.

2# Write a C program to perform the following operations:

a. Insert an element into an AVL tree.

b. Delete an element from an AVL tree.

c. Search for a key element in an AVL tree.

WEEK #14

Objectives

- ❒ *To learn the linked list applications.*

Outcomes

After completing this, the students would be able to:

- ❒ *Learn the implementation of various functions of a hash table using linked list.*

Problems

1# Write a C program to implement the following functions of a hash table (dictionary) using Linked List.

a) Insertion

b) Deletion

c) Search

d) Hash Function